

MATERIAL DE APOYO — CLASE 2

De la naturaleza del software al proceso de construirlo: ingeniería, proceso y modelos

ISO1 (Ecom-82) — Bloque 1: Fundamentos (continuación)

1. Lectura breve para trabajar en grupo (15 min)

Cerrando la idea de "naturaleza del software": las webapps

En la Clase 1 distinguimos dominios de aplicación: sistemas de información, sistemas embebidos, aplicaciones móviles y sistemas con inteligencia artificial. Falta un dominio con una naturaleza particular: las aplicaciones web. Las webapps no son simplemente "programas que corren en un navegador". Tienen rasgos propios que las distinguen de otros dominios:

- **Evolución continua:** a diferencia del software empaquetado tradicional, una webapp puede actualizarse varias veces por semana, incluso por día, sin que el usuario instale nada.
- **Naturaleza distribuida y sin estado:** cada solicitud al servidor es independiente; sostener una "sesión" o una experiencia coherente es una decisión de diseño, no algo automático.
- **Estética y usabilidad como requisito funcional:** en una webapp, la interfaz no es un "extra": es, para buena parte de sus usuarios, la totalidad de su experiencia con el sistema.
- **Exposición y seguridad:** al estar accesible públicamente por diseño, la superficie de ataque es mayor que la de un sistema interno.

Esto confirma algo que ya intuíamos en la Clase 1: no hay una única "forma correcta" de hacer ingeniería de software. El dominio de aplicación condiciona qué riesgos importan más.

¿Qué es, formalmente, la Ingeniería de Software?

Ya trabajamos qué es el software (programas + estructuras de datos + documentación) y por qué la falta de documentación lo vuelve frágil. Ahora demos el paso siguiente: ¿qué es la disciplina que se ocupa de construirlo bien?

La Ingeniería de Software es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, la operación y el mantenimiento del software — y el estudio de esos enfoques. La palabra clave es "enfoque": no se trata de escribir código con buena intención, sino de seguir un proceso reconocible, con actividades identificables, que se pueda repetir, medir y mejorar.

Esta definición importa porque marca la diferencia entre "programar" y "hacer ingeniería": cualquiera puede escribir instrucciones que funcionen una vez; la ingeniería busca que funcionen de manera sostenida, sean mantenibles, y que el conocimiento detrás de las decisiones no dependa de una sola persona (tal como vimos con el caso de la startup en la Actividad 4 de la Clase 1).

El proceso del software: el "cómo" detrás del "qué"

Todo proyecto de software, sin importar el dominio o la metodología elegida, atraviesa un conjunto de actividades genéricas. Este es el marco de trabajo (framework) que se repite, en mayor o menor medida, en cualquier proceso de desarrollo:

- **Comunicación:** entender el problema real y a quién le pertenece, antes de tocar una sola línea de código.
- **Planeación:** trazar un mapa del trabajo por hacer — tareas, riesgos, recursos, cronograma.
- **Modelado:** construir representaciones (requisitos, diseño) que permitan razonar sobre el sistema antes de construirlo.
- **Construcción:** generar el código y verificarlo mediante pruebas.
- **Despliegue:** entregar el producto, recoger retroalimentación real del usuario, y ajustar en función de ella.

Estas cinco actividades no son "pasos" que se hacen una sola vez y en ese orden estricto: son actividades genéricas que reaparecen, con distinta intensidad, en cada modelo de proceso — ya sea uno secuencial o uno iterativo.

La práctica de la Ingeniería de Software: esencia y principios

Más allá del marco de actividades, hay una "esencia" común a la práctica profesional, sin importar el modelo de proceso elegido:

- **Comprender el problema antes de proponer una solución:** ¿quién tiene el problema?, ¿qué datos y funciones se requieren?, ¿puede representarse de forma más simple?
- **Planear una solución:** identificar patrones o soluciones ya conocidas que se puedan adaptar, en lugar de reinventar todo desde cero.
- **Ejecutar el plan:** el diseño y el código deben poder rastrearse hasta los requisitos que les dieron origen.
- **Examinar el resultado con sentido crítico:** cada incremento de software debe probarse, y el equipo debe estar dispuesto a revisar sus propias decisiones.

Y como principio general que atraviesa todo lo anterior: el software de calidad reduce el costo de mantenerlo. Un sistema difícil de entender, sin documentación, o con decisiones no justificadas, no es "más rápido de entregar hoy": es una deuda que alguien va a pagar mañana — exactamente la misma idea que trabajamos con la documentación en la Clase 1.

2. Modelos de proceso: distintas formas de organizar esas actividades

Un modelo general de proceso

Ninguna organización aplica las cinco actividades genéricas siempre de la misma manera. Por eso existen patrones de proceso — soluciones ya probadas frente a problemas recurrentes de organización del trabajo — y mecanismos de evaluación y mejora del proceso: revisar, después de cada proyecto, qué funcionó y qué no, para ajustar cómo se trabaja la próxima vez. Un equipo que nunca evalúa su propio proceso tiende a repetir los mismos errores proyecto tras proyecto.

Modelos de proceso prescriptivos

Los modelos prescriptivos proponen una secuencia definida y ordenada para las actividades genéricas. Los más relevantes para nuestra materia:

- **Modelo en cascada:** las actividades se ejecutan de forma lineal y secuencial (comunicación → planeación → modelado → construcción → despliegue), cada una completándose antes de iniciar la siguiente. Funciona bien cuando los requisitos son estables y bien comprendidos desde el inicio; es rígido frente al cambio.
- **Modelos incrementales:** combinan elementos lineales e iterativos: se entrega un primer incremento funcional (un subconjunto de funciones) y luego se van agregando incrementos sucesivos, cada uno más completo que el anterior. Útil cuando hace falta poner algo funcional en manos del usuario pronto.
- **Modelos evolutivos:** asumen que el software crece y cambia con el tiempo, y que no siempre es posible definir todos los requisitos por adelantado. Construyen versiones cada vez más completas del sistema, incorporando la retroalimentación del cliente en cada vuelta.
- **Modelos concurrentes:** representan las actividades como estados en paralelo (en desarrollo, en espera de cambios, en revisión, terminada) en lugar de una secuencia única; son especialmente relevantes cuando distintas partes del sistema avanzan a ritmos diferentes.

Ningún modelo es "el correcto" en abstracto: la elección depende de qué tan estables son los requisitos, qué tan crítico es el sistema, y cuánto puede esperar el usuario para ver resultados — la misma lógica de "depende del contexto" que ya aplicamos a los dominios de aplicación en la Clase 1.

Fuente conceptual: Pressman, R. Ingeniería del Software, un enfoque práctico; Sommerville, I. Ingeniería de Software. Adaptado y contextualizado para ISO1, Universidad Columbia del Paraguay, 2026.